native structures for tuples; in other languages you can make do by returning an array with 2 elements.

In modern programming languages, in fact, you can typically speed up these operations by using fixed size numerical arrays of primitive types and vector algebra. The price to pay is that, when we get a request to access the `i`-th bit in the filter, we need to extract two coordinates from index `i`: which array element is storing the `i`-th bit, and what's the offset of the bit we need to extract with respect to that element.

Listing 4.4 shows what this means and how it can be computed.

Once we have those two indices, we can easily read or write any bit, it just becomes a matter of bitwise arithmetic. For instance, listing 4.5 shows the `readBit` method taking care of the reading part.

Listing 4.5 Method readBit

```
function readBit(bitsArray,
index)                                              #1
  (element, bit) ← findBitCoordinates(bitsArray,
index)                          #2
  return (bitsArray[element] & (1 << bit)) >>
bit                               #3
```

#1 Method `readBit` extracts the `index`-th bit from the bits array passed as first argument. It returns the value of the bit, so either 0 or `1`.

#2 Retrieve the element index and offset for the bit in the bits array.